

Transformer Encoders and Decoders

A practical PDF workbook with concepts, exercises, mini labs, and a reusable dataset for training and inference.

Audience: learners who know basic Python and want to understand how transformer encoder, decoder, and encoder-decoder architectures are used.

Deliverables: This workbook includes a dataset appendix. A separate CSV file named **transformer_workbook_dataset.csv** is provided for hands-on use.

Module	Focus	Hands-on outcome
1	Transformer building blocks	Explain tokens, embeddings, attention, residual paths
2	Encoders	Use contextual representations for classification
3	Decoders	Use masked self-attention for next-token generation
4	Encoder-decoder models	Map source sequences to target sequences
5	Dataset and experiments	Train/evaluate/infer with a toy dataset

Table of Contents

- How to use this workbook
- Transformer mental model
- Self-attention from inputs to outputs
- Encoder-only transformers
- Decoder-only transformers
- Encoder-decoder transformers
- Training workflow
- Inference workflow
- Mini labs
- Dataset reference

How to Use This Workbook

Work through the sections in order. Each section gives a concept explanation, a quick check, and a hands-on prompt. Use the included CSV dataset for experiments. The examples are intentionally small so that the mechanics are visible.

Recommended setup: Python, pandas, PyTorch or another deep learning framework, and optionally a transformer library. The workbook is framework-neutral, but the mini labs use common terms such as tokenizer, model, train split, validation split, logits, and generated text.

Learning goals

- Distinguish encoder, decoder, and encoder-decoder transformer architectures.
- Explain self-attention, masked self-attention, and cross-attention.
- Prepare a dataset for classification, generation, summarization, or translation-style tasks.
- Describe what changes between training and inference.
- Run small experiments and interpret their outputs.

Checkpoint questions

1. What does a tokenizer produce?
2. Why do transformers need position information?
3. What is the difference between training and inference?

1. Transformer Mental Model

A transformer is a neural network architecture that processes tokens using attention. Instead of reading a sequence strictly left to right, attention lets tokens compare themselves with other tokens and decide what information matters.

Core pipeline

Stage	What happens	Typical artifact
Tokenization	Text is split into model-readable pieces.	token ids
Embedding	Token ids become dense vectors.	embedding matrix rows
Position signal	Order information is added.	position embeddings or rotary features
Attention	Tokens exchange information.	contextual vectors
Feed-forward	Each token vector is transformed.	hidden states
Prediction head	Hidden states become task outputs.	class logits or token logits

Key idea: the same attention operation can support understanding, generation, or sequence-to-sequence mapping depending on masking and architecture.

Exercise 1A: Label the pipeline

For the sentence "The decoder generated a summary", list a plausible sequence of stages from raw text to model prediction. Leave room for token ids, embeddings, attention, and output logits.

2. Self-Attention From Inputs to Outputs

Self-attention builds a weighted mixture of token representations. Each token creates a query, a key, and a value. The query asks what the token is looking for, the keys describe what each token offers, and the values carry the information to be mixed.

Symbol	Meaning	Plain-English role
Q	Queries	What each token is searching for
K	Keys	What each token can match against
V	Values	What information gets passed forward
$\text{softmax}(QK^T)$	Attention weights	How strongly tokens attend to each other

Scaled dot-product attention:

```
scores = (Q @ K.T) / sqrt(d_k)
weights = softmax(scores + mask)
context = weights @ V
```

Multi-head attention repeats this operation in parallel with different learned projections. Each head can learn a different pattern, such as local syntax, long-distance references, or task-specific cues.

Exercise 2A: Explain the mask

A batch contains one short sentence and one long sentence. Why should padding tokens be masked?

Exercise 2B: Interpret attention

Suppose the word "it" attends strongly to "model". What relationship might the attention head be capturing?

3. Encoder-Only Transformers

An encoder reads the full input sequence at once. Because it can attend bidirectionally, token representations can use information from both left and right context. This makes encoders strong for understanding tasks.

Used for	Output style	Example from dataset
Classification	One label for an input	encoder_classification rows
Token tagging	One label per token	named entity recognition
Embeddings/retrieval	Vector representation	semantic search
Reranking	Score for input pairs	query-document relevance

Typical classification flow

text -> tokenizer -> encoder hidden states -> pooled vector -> classifier head -> label logits

Mini lab: encoder classifier

- 1 Load rows where task_type equals encoder_classification.
- 2 Use input_text as the model input and label as the target.
- 3 Split by the split column: train, validation, test.
- 4 Train a classifier and evaluate accuracy on the test split.
- 5 Run inference on: "The model produced useful summaries."

Reflection

Why is a bidirectional encoder suitable for sentiment or topic classification?

4. Decoder-Only Transformers

A decoder generates tokens from left to right. Causal masking prevents each position from attending to future target tokens. This matches the next-token prediction objective: given the previous tokens, predict the next token.

Training input	Target	Mask
Prompt plus previous target tokens	Next target token at each position	Causal mask hides future tokens
Example: "Write one sentence about self-attention"	Generated sentence	Future words are hidden during training

Training versus inference

Mode	What the decoder sees	Common method
Training	Gold previous tokens	teacher forcing
Inference	Its own generated tokens	greedy, sampling, or beam search

Mini lab: decoder generation

- 1 Load rows where task_type equals decoder_generation.
- 2 Format each row as prompt plus target_text.
- 3 Train or fine-tune a small causal language model on the training rows.
- 4 At inference, provide only the prompt and ask the model to continue.
- 5 Compare greedy decoding with sampling.

Exercise 4A

In one paragraph, explain why a decoder should not see future target tokens during training.

5. Encoder-Decoder Transformers

An encoder-decoder transformer combines an encoder for the source sequence and a decoder for the target sequence. The decoder uses masked self-attention over generated target tokens and cross-attention over encoder outputs.

Component	Attention type	Purpose
Encoder	Bidirectional self-attention	Build source representations
Decoder self-attention	Causal self-attention	Track generated target prefix
Decoder cross-attention	Attention over encoder outputs	Condition generation on source input

Good fit tasks

- Summarization: long source text to shorter target text.
- Translation: source language or format to target language or format.
- Structured generation: free text to code, SQL, labels, or normalized forms.
- Instruction-conditioned rewriting: source text plus an instruction to target output.

Mini lab: sequence-to-sequence

- 1 Load rows where task_type starts with encoder_decoder.
- 2 Use input_text as the encoder source and target_text as the decoder target.
- 3 Train with teacher forcing.
- 4 At inference, encode the source once and decode target tokens step by step.
- 5 Evaluate exact match for pseudo-code rows and qualitative quality for summaries.

Exercise 5A

Mark each task as encoder-only, decoder-only, or encoder-decoder: spam classification, article summarization, story continuation, translation, retrieval embedding.

6. Training Workflow

Training teaches model parameters to reduce loss on examples. The exact loss depends on the task: classification cross-entropy for encoder classifiers, causal language modeling loss for decoders, and sequence-to-sequence token loss for encoder-decoder models.

Task	Input column	Target column	Metric idea
encoder_classification	input_text	label	accuracy or F1
decoder_generation	input_text + target_text	next tokens	loss or sample quality
encoder_decoder_summarization	input_text	target_text	ROUGE-style overlap or human review
encoder_decoder_translation	input_text	target_text	exact match or semantic match

Baseline training checklist

- Start with a tiny subset and overfit it. This catches data formatting problems.
- Check that labels and targets are not empty for the selected task.
- Keep train, validation, and test examples separate.
- Track training loss and validation performance.
- Save the tokenizer, model weights, and label mapping together.

Example data loading

```
import pandas as pd

df = pd.read_csv("transformer_workbook_dataset.csv")
train = df[df.split == "train"]
valid = df[df.split == "validation"]
test = df[df.split == "test"]

enc = train[train.task_type == "encoder_classification"]
seq2seq = train[train.task_type.str.startswith("encoder_decoder")]
dec = train[train.task_type == "decoder_generation"]
```

Exercise 6A

Write a validation rule that checks whether every classification row has a non-empty label and every generation row has a non-empty target_text.

7. Inference Workflow

Inference uses a trained model to produce outputs. Encoders usually return a label or vector in one forward pass. Decoders and encoder-decoder models usually generate tokens iteratively.

Architecture	Inference pattern	Example output
Encoder-only	Single forward pass	positive
Decoder-only	Autoregressive loop	generated continuation
Encoder-decoder	Encode once, decode step by step	summary or translated target

Decoding options

Method	Idea	Tradeoff
Greedy	Pick highest-probability token each step	Fast but can be dull or repetitive
Sampling	Randomly sample from a probability distribution	More varied but less predictable
Beam search	Keep multiple candidate sequences	Often stronger but slower

Inference prompts to try

- Encoder: "The summary was concise and correct."
- Decoder: "Write one sentence about cross-attention."
- Encoder-decoder: "Transformers compare tokens with attention and use masks for generation."

Exercise 7A

Describe one risk of using the training target text during inference.

8. Mini Projects

Choose one project and complete it using the dataset. Each project can be done with a pretrained model, a small custom model, or even a rule-based baseline before training.

Project	Goal	Minimum deliverable
A. Encoder classifier	Predict sentiment-like labels	Test accuracy and three example predictions
B. Decoder prompt responder	Generate one-sentence explanations	Three prompts and generated completions
C. Seq2seq summarizer	Summarize transformer concept paragraphs	Three source-target-output comparisons
D. Seq2seq pseudo-code translator	Map instructions to pseudo-code	Exact-match score and error analysis

Experiment log template

Field	Your notes
Model	
Task rows used	
Tokenizer settings	
Epochs	
Learning rate	
Metric	
Best result	
Most interesting error	

9. Dataset Reference

The dataset is intentionally small and educational. It is not meant to produce a production-quality model. Its purpose is to help you test end-to-end training and inference pipelines without needing a large external corpus.

CSV schema

Column	Description
id	Stable row identifier
task_type	One of encoder_classification, decoder_generation, encoder_decoder_summarization, encoder_decoder_translation
split	train, validation, or test
input_text	Source text, prompt, or classification input
target_text	Generation target for decoder and encoder-decoder rows
label	Classification label for encoder rows

Sample rows

id	task_type	split	input_text	target_text	label
001	encoder_classification	train	The model finished training faster than expected.		positive
002	encoder_classification	train	The validation loss kept increasing after epoch three.		negative
003	encoder_classification	train	The tokenizer loaded with the default settings.		neutral
004	encoder_classification	train	The inference results were accurate and consistent.		positive
015	encoder_decoder_summarization	train	A decoder generates tokens one at a time while masking future position...	Decoders generate masked next tokens for text generatio...	
016	encoder_decoder_summarization	train	An encoder-decoder model first encodes the source text and then decode...	Encoder-decoder models support sequence-to-sequence gen...	
017	encoder_decoder_summarization	train	Teacher forcing trains a decoder by providing the previous gold target...	Teacher forcing uses gold previous tokens during decode...	
018	encoder_decoder_summarization	train	Positional information is needed because attention alone does not know...	Position signals help attention understand token order.	
035	decoder_generation	train	Write one sentence about inference.	Inference uses a trained model to produce predictions o...	
036	decoder_generation	train	Write one sentence about cross-attention.	Cross-attention lets a decoder attend to encoder output...	
037	decoder_generation	validation	Write one sentence about evaluation.	Evaluation measures how well a model performs on data i...	
038	decoder_generation	test	Write one sentence about tokenization.	Tokenization splits text into units such as words, subw...	

Suggested preprocessing

- Filter by task_type before training so input and target fields are consistent.
- For classification, map labels such as positive, neutral, negative to integer ids.
- For generation, keep target_text as the supervised sequence.

- Do not train on validation or test rows.
- Add your own rows to make the task less toy-like.

Appendix A: Blank Worksheet

Architecture chosen and why

Data fields used

Training settings

Inference examples

Errors and improvements
